ECE 526L Spring 2025 Lab 2 Design a Full Adder

2/13/2025

Muhammad Umar
CSUN ID: 204533244

Instructor: Prof. Orod Haghighiara

I hereby attest that this lab report is entirely my own work. I have not copied either code or text from anyone, nor have I allowed or will I allow anyone to copy my work.

Name (printed): **Muhammad Umar**

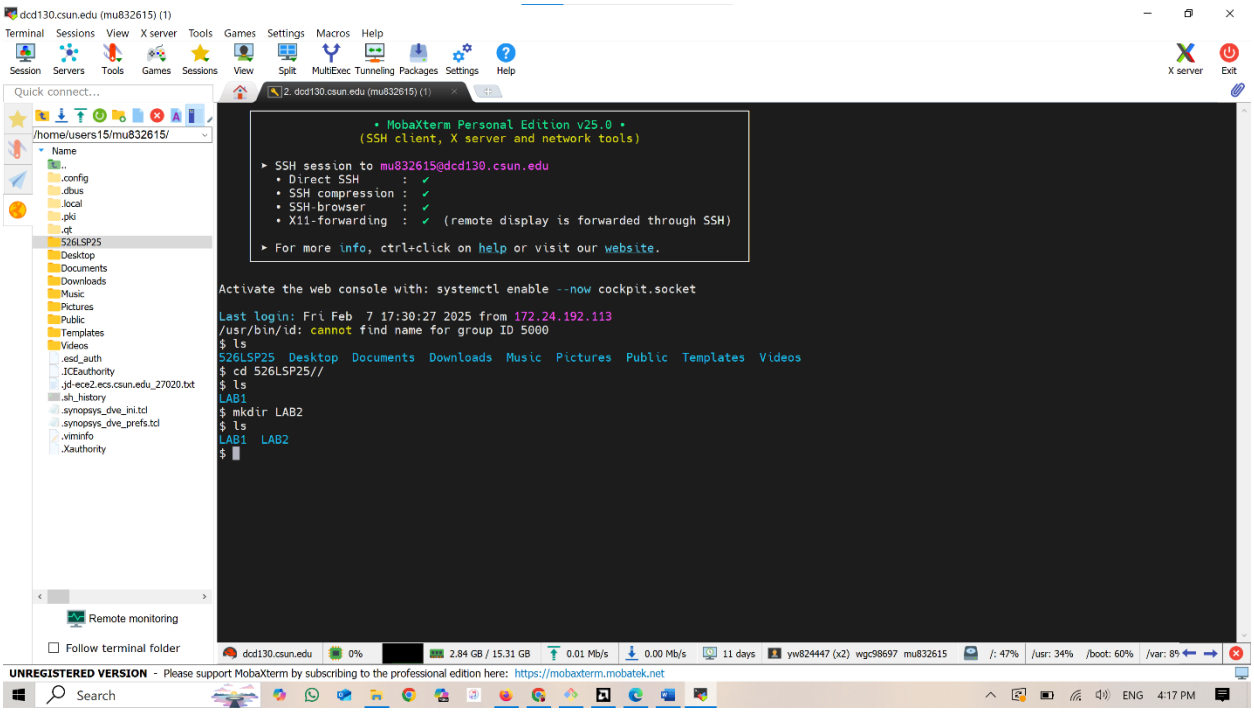
Name (signed):  Date: 02/13/2025

Introduction

This introductory lab objective is to familiarize ourselves with using digital design concepts by implementing and testing basic combinational circuits using Verilog. The primary objective is to design and verify a **half adder** and **full adder** at the gate level and how to instantiate the design in the testbench for verification.

Connecting to the CSUN Server

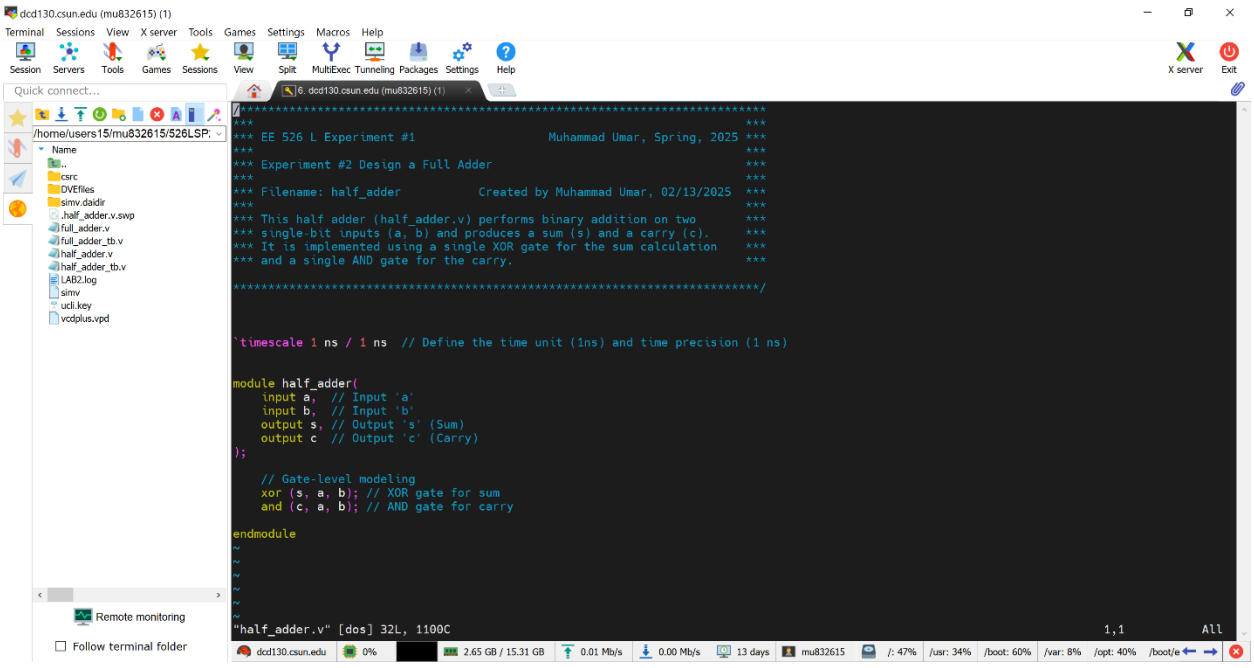
Before starting the lab, we need to connect to a CSUN server using VPN and then open **MobaXterm**. The available machines range from 130 -159 and I have selected **dcd130**. After connecting, we navigate to the working directory and create a directory for the lab as mentioned in the below snapshot.



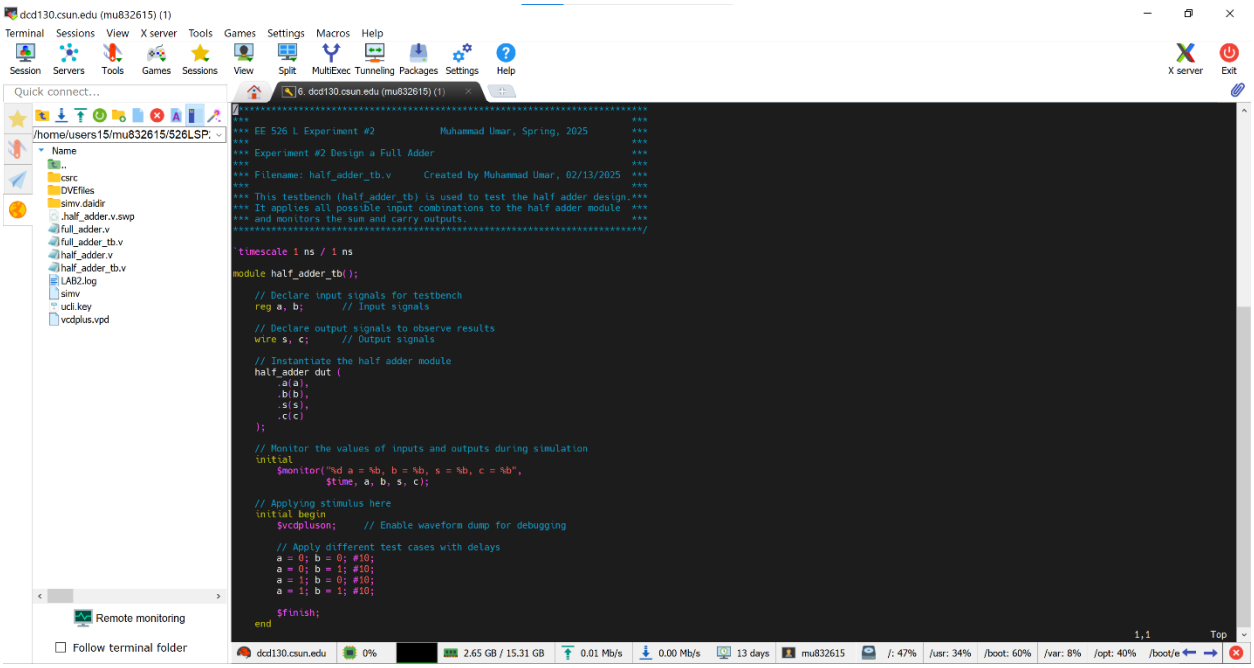
Writing Verilog Code for Half Adder and Full Adder using Vim

I used a text editor called Vim for writing Verilog code and we can access it by using the command **vi half_adder.v**.

Half Adder Design (half_adder.v)



Half Adder Testbench (half_adder_tb.v)



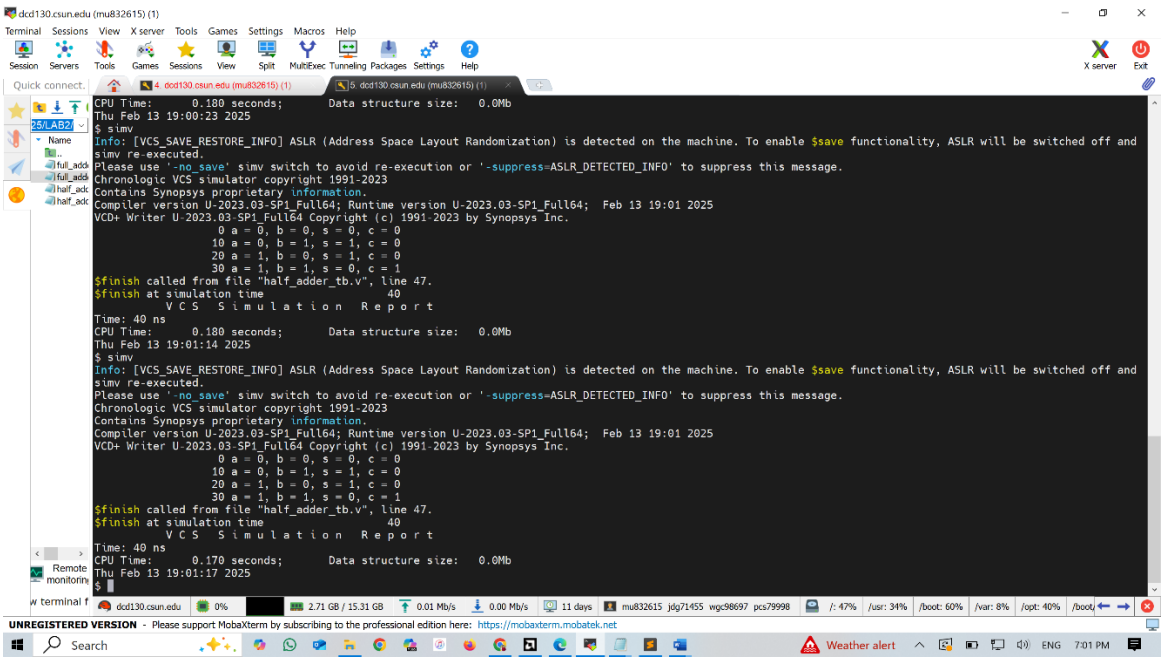
Compiling and Simulating Using VCS

Now compiling the design and testbench file using the following command:

vcs -debug_access+all half_adder.v half_adder_tb.v

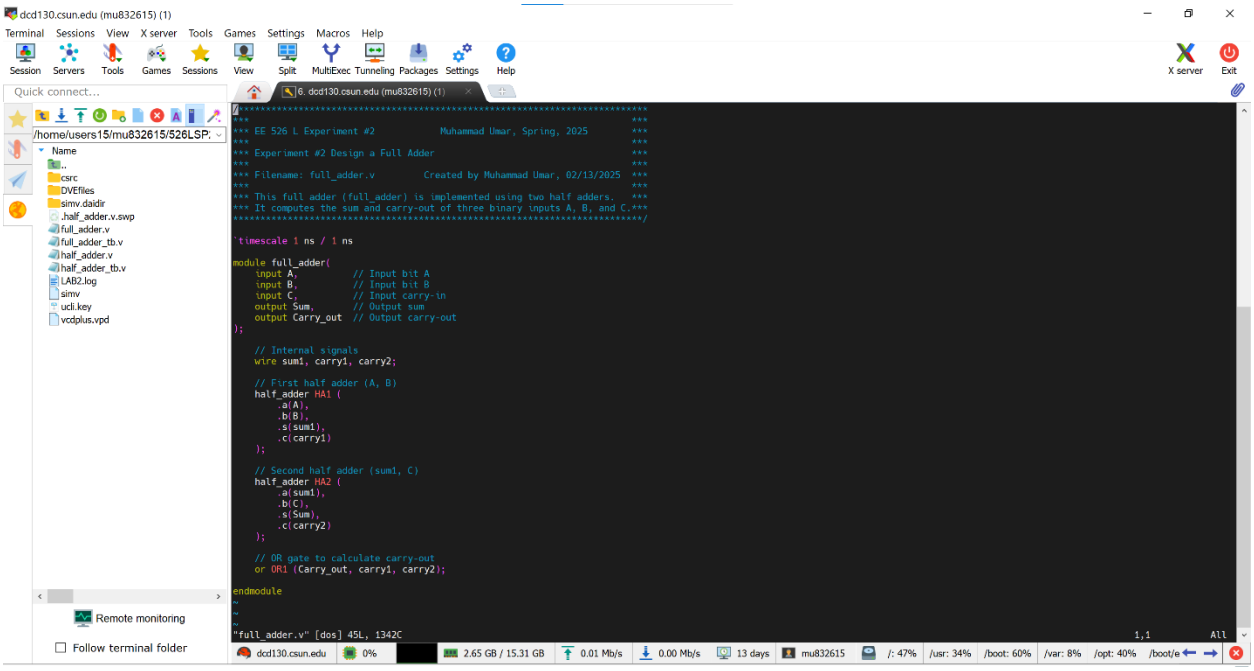
Invoking VCS Simulator

By using **simv** command



Full Adder Design (full_adder.v)

By using prior half adder and instantiating it two times with different instance.



```
/*****************************************************************************
***                                                                        ***
*** EE 526 L Experiment #2           Muhammad Umar, Spring, 2025        ***
***                                                                        ***
*** Experiment #2 Design a Full Adder                                     ***
***                                                                        ***
*** Filename: full_adder.v         Created by Muhammad Umar, 02/13/2025  ***
***                                                                        ***
*** This full adder (full_adder) is implemented using two half adders.    ***
*** It computes the sum and carry-out of three binary inputs A, B, and C.***
*****************************************************************************/

`timescale 1 ns / 1 ns

module full_adder(
    input A,          // Input bit A
    input B,          // Input bit B
    input C,          // Input carry-in
    output Sum,       // Output sum
    output Carry_out  // Output carry-out
);

    // Internal signals
    wire sum1, carry1, carry2;

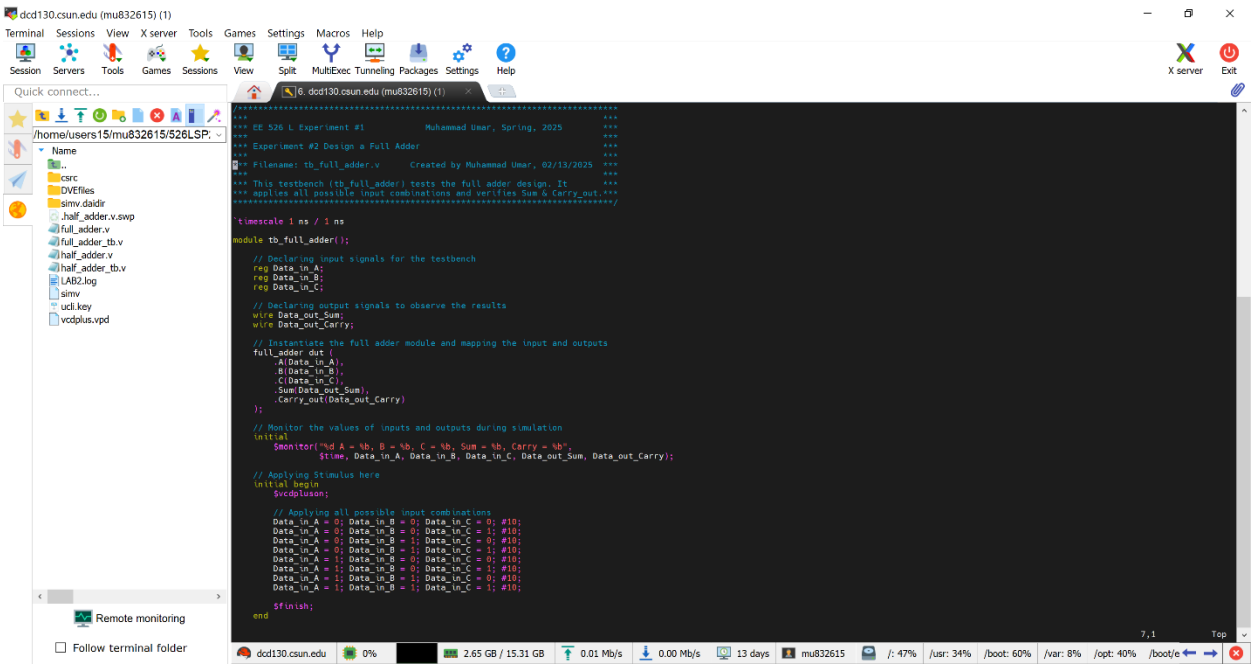
    // First half adder (A, B)
    half_adder HA1 (
        .a(A),
        .b(B),
        .s(sum1),
        .c(carry1)
    );

    // Second half adder (sum1, C)
    half_adder HA2 (
        .a(sum1),
        .b(C),
        .s(Sum),
        .c(carry2)
    );

    // OR gate to calculate carry-out
    or OR1 (Carry_out, carry1, carry2);

endmodule
```

Full Adder Testbench (full_adder_tb.v)



```
/*****************************************************************************
***                                                                        ***
*** EE 526 L Experiment #1           Muhammad Umar, Spring, 2025        ***
***                                                                        ***
*** Experiment #2 Design a Full Adder                                     ***
***                                                                        ***
*** Filename: tb_full_adder.v      Created by Muhammad Umar, 02/13/2025  ***
***                                                                        ***
*** This testbench (tb_full_adder) tests the full adder design. It        ***
*** applies all possible input combinations and verifies Sum & Carry_out.***
*****************************************************************************/

`timescale 1 ns / 1 ns

module tb_full_adder();

    // Declaring input signals for the testbench
    reg Data_in_A;
    reg Data_in_B;
    reg Data_in_C;

    // Declaring output signals to observe the results
    wire Data_out_Sum;
    wire Data_out_Carry;

    // Instantiate the full adder module and mapping the input and outputs
    full_adder dut (
        .A(Data_in_A),
        .B(Data_in_B),
        .C(Data_in_C),
        .Sum(Data_out_Sum),
        .Carry_out(Data_out_Carry)
    );

    // Monitor the values of inputs and outputs during simulation
    initial
        $monitor("%d A = %b, B = %b, C = %b, Sum = %b, Carry = %b",
                 $time, Data_in_A, Data_in_B, Data_in_C, Data_out_Sum, Data_out_Carry);

    // Applying Stimulus here
    initial begin
        $vcdplusson;

        // Applying all possible input combinations
        Data_in_A = 0; Data_in_B = 0; Data_in_C = 0; #10;
        Data_in_A = 0; Data_in_B = 0; Data_in_C = 1; #10;
        Data_in_A = 0; Data_in_B = 1; Data_in_C = 0; #10;
        Data_in_A = 0; Data_in_B = 1; Data_in_C = 1; #10;
        Data_in_A = 1; Data_in_B = 0; Data_in_C = 0; #10;
        Data_in_A = 1; Data_in_B = 0; Data_in_C = 1; #10;
        Data_in_A = 1; Data_in_B = 1; Data_in_C = 0; #10;
        Data_in_A = 1; Data_in_B = 1; Data_in_C = 1; #10;

        $finish;
    end
```

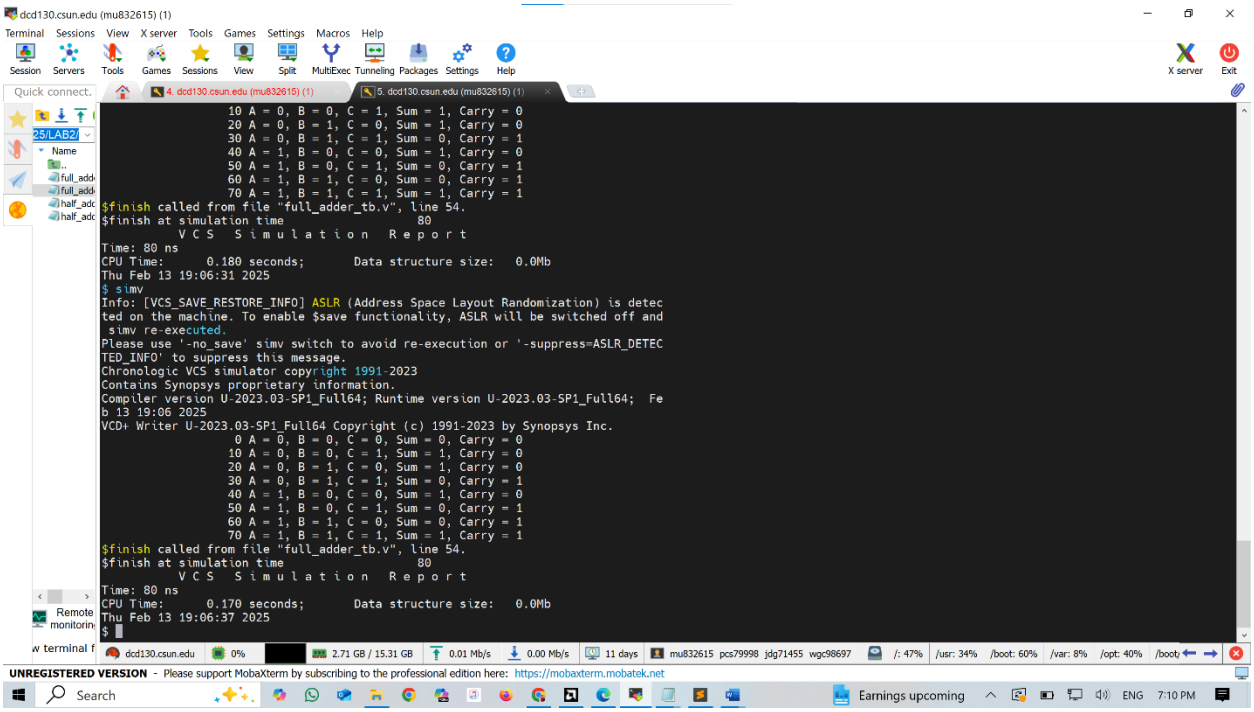
Compiling and Simulating Using VCS

Now compiling the design and testbench file using the following command:

vcs -debug_access+all half_adder.v full_adder.v full_adder_tb.v

Invoking VCS Simulator

By using **simv** command



Creating Log File

We have to use the following command for log file generation.

simv –l LAB2.log

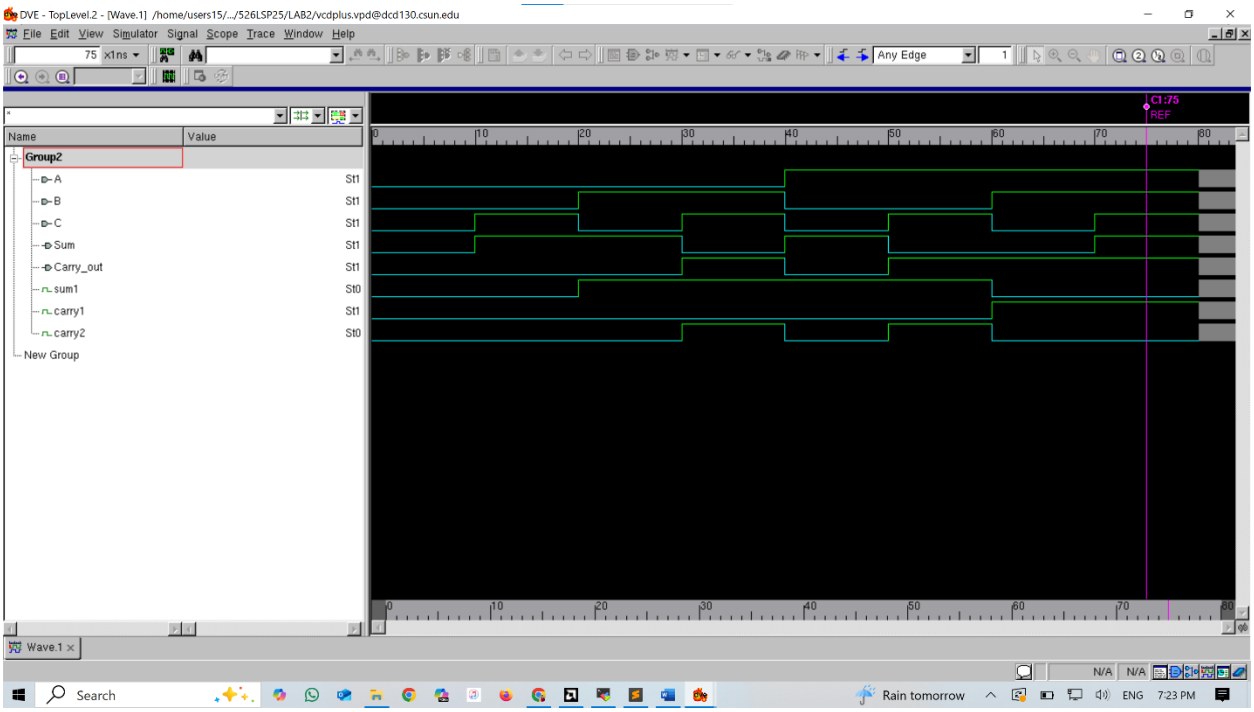
Waveform

We have to use the following command to display dve and add vpd file for dumping signals in a waveform.

dve -full64

Adding Signals to Waveform

We have to add signals by selecting all signals including the internal signals of the module by getting it from the module after double-clicking it and then adding all signals to the New Wave View.



Source Code

Design File (half_adder.v)

```
/*************************************************************************
***                                                                    ***
*** EE 526 L Experiment #1                  Muhammad Umar, Spring, 2025 ***
***                                                                    ***
*** Experiment #2 Design a Full Adder                                  ***
***                                                                    ***
*** Filename: half_adder     Created by Muhammad Umar, 02/13/2025  ***
***                                                                     ***
*** This half adder (half_adder.v) performs binary addition on two     ***
*** single-bit inputs (a, b) and produces a sum (s) and a carry (c).    ***
*** It is implemented using a single XOR gate for the sum calculation  ***
```

*** and a single AND gate for the carry. ***

**********/

```
`timescale 1 ns / 1 ns  // Define the time unit (1ns) and time precision (1 ns)

module half_adder(
   input a,  // Input 'a'
   input b,  // Input 'b'
   output s, // Output 's' (Sum)
   output c  // Output 'c' (Carry)
);

   // Gate-level modeling
   xor (s, a, b); // XOR gate for sum
   and (c, a, b); // AND gate for carry

endmodule
```

Testbench File(half_adder_tb.v)

```
/**********************************************************************
*** EE 526 L Experiment #2          Muhammad Umar, Spring, 2025                ***
*** Experiment #2 Design a Full Adder                                          ***
*** Filename: half_adder_tb.v      Created by Muhammad Umar, 02/13/2025        ***
***                                                                           ***
*** This testbench (half_adder_tb) is used to test the half adder design.     ***
*** It applies all possible input combinations to the half adder module       ***
*** and monitors the sum and carry outputs.                                   ***
**********/
```

```
`timescale 1 ns / 1 ns

module half_adder_tb();

  // Declare input signals for testbench
  reg a, b;       // Input signals

  // Declare output signals to observe results
  wire s, c;      // Output signals

  // Instantiate the half adder module
  half_adder dut (
    .a(a),
    .b(b),
    .s(s),
    .c(c)
  );

  // Monitor the values of inputs and outputs during simulation
  initial
    $monitor("%d a = %b, b = %b, s = %b, c = %b",
          $time, a, b, s, c);

  // Applying stimulus here
  initial begin
    $vcdpluson;// Enable waveform dump for debugging

    // Apply different test cases with delays
    a = 0; b = 0; #10;
    a = 0; b = 1; #10;
    a = 1; b = 0; #10;
    a = 1; b = 1; #10;
```

```
    $finish;
  end

endmodule
```

Design File (full_adder.v)

```
/*************************************************************************
*** EE 526 L Experiment #2          Muhammad Umar, Spring, 2025              ***
*** Experiment #2 Design a Full Adder                                        ***
*** Filename: full_adder.v        Created by Muhammad Umar, 02/13/2025       ***
***                                                                          ***
*** This full adder (full_adder) is implemented using two half adders.       ***
*** It computes the sum and carry-out of three binary inputs A, B, and C     .***
*************************************************************************/

`timescale 1 ns / 1 ns

module full_adder(
  input A,        // Input bit A
  input B,        // Input bit B
  input C,        // Input carry-in
  output Sum,     // Output sum
  output Carry_out  // Output carry-out
);

  // Internal signals
  wire sum1, carry1, carry2;

  // First half adder (A, B)
```

```
    half_adder HA1 (
        .a(A),
        .b(B),
        .s(sum1),
        .c(carry1)
    );

    // Second half adder (sum1, C)
    half_adder HA2 (
        .a(sum1),
        .b(C),
        .s(Sum),
        .c(carry2)
    );

    // OR gate to calculate carry-out
    or OR1 (Carry_out, carry1, carry2);

endmodule
```

Testbench File(full_adder_tb.v)

```
/*************************************************************************
*** EE 526 L Experiment #2          Muhammad Umar, Spring, 2025          ***
***                                                                      ***
*** Experiment #2 Design a Full Adder                                    ***
*** Filename: tb_full_adder.v      Created by Muhammad Umar, 02/13/2025  ***
***                                                                      ***
*** This testbench (tb_full_adder) tests the full adder design. It       ***
*** applies all possible input combinations and verifies Sum & Carry_out. ***
*************************************************************************/
```

```
`timescale 1 ns / 1 ns

module tb_full_adder();

  // Declaring input signals for the testbench
  reg Data_in_A;
  reg Data_in_B;
  reg Data_in_C;

  // Declaring output signals to observe the results
  wire Data_out_Sum;
  wire Data_out_Carry;

  // Instantiate the full adder module and mapping the input and outputs
  full_adder dut (
    .A(Data_in_A),
    .B(Data_in_B),
    .C(Data_in_C),
    .Sum(Data_out_Sum),
    .Carry_out(Data_out_Carry)
  );

  // Monitor the values of inputs and outputs during simulation
  initial
    $monitor("%d A = %b, B = %b, C = %b, Sum = %b, Carry = %b",
          $time, Data_in_A, Data_in_B, Data_in_C, Data_out_Sum, Data_out_Carry);

  // Applying Stimulus here
  initial begin
    $vcdpluson;

    // Applying all possible input combinations
```

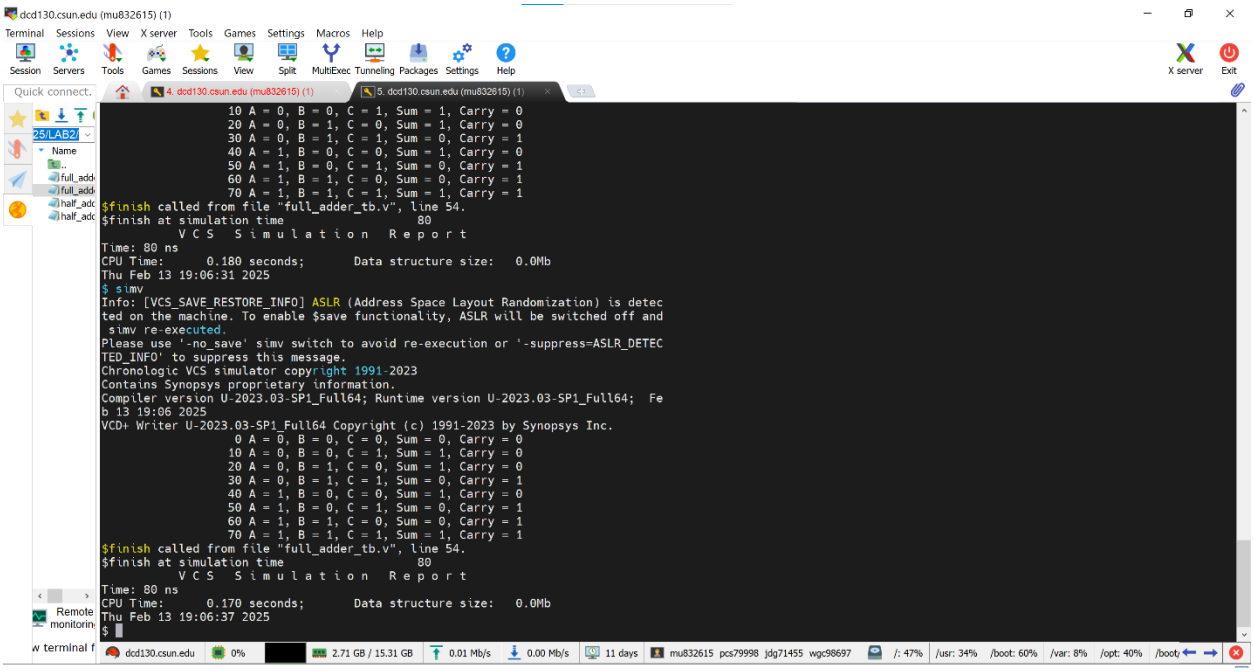
```
    Data_in_A = 0; Data_in_B = 0; Data_in_C = 0; #10;
    Data_in_A = 0; Data_in_B = 0; Data_in_C = 1; #10;
    Data_in_A = 0; Data_in_B = 1; Data_in_C = 0; #10;
    Data_in_A = 0; Data_in_B = 1; Data_in_C = 1; #10;
    Data_in_A = 1; Data_in_B = 0; Data_in_C = 0; #10;
    Data_in_A = 1; Data_in_B = 0; Data_in_C = 1; #10;
    Data_in_A = 1; Data_in_B = 1; Data_in_C = 0; #10;
    Data_in_A = 1; Data_in_B = 1; Data_in_C = 1; #10;

    $finish;
  end

endmodule
```

RESULT

After simulating the design and verifying it through waveform and log file, we can see from the snapshot that there are no errors noted and results shown below.



Conclusion

We have compiled a Verilog module for a **half adder** and **full adder,** instantiating half adder twice with different instance name and make it the full adder module, checks its functionality using a test bench module by using Synopsys VCS simulator. The simulation results confirmed that the full adder operated correctly according to the provided stimulus.